

TRACE

AI-Powered Multi-Camera Person Tracking & Re-Identification

Software Engineering Laboratory

Assignment 2

SRS Document | DFD Diagrams | Class Diagram | UML Diagrams

Student Name: _____

Roll Number: _____

Date: 2 October 2026

Table of Contents

1. Software Requirements Specification (SRS)
 - 1.1 Introduction | 1.2 Overall Description | 1.3 Functional Requirements
 - 1.4 Non-Functional Requirements | 1.5 External Interface Requirements
 - 1.6 Data Requirements | 1.7 System Constraints | 1.8 Assumptions | 1.9 Future Scope
2. DFD Diagrams
 - 2.1 Level 0 (Context) | 2.2 Level 1 | 2.3 Level 2: Video Processing
 - 2.4 Level 2: Matching | 2.5 Level 2: Route Reconstruction
3. UML Diagrams
 - 3.1 Use Case | 3.2 Activity | 3.3 Sequence | 3.4 Component
4. Class Diagram
5. Traceability Matrix
6. References

Software Requirements Specification

Trace — AI-Powered Multi-Camera Person Tracking & Re-Identification

Software Engineering Laboratory — Assignment 2

Field	Value
Project Title	Trace
Subtitle	AI-Powered Multi-Camera Person Tracking & Re-Identification
Course	Software Engineering Laboratory
Assignment	Assignment 2
Student Name	(placeholder)
Roll Number	(placeholder)
Date	2 October 2026
Version	1.0

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. Functional Requirements
- 4. Non-Functional Requirements
- 5. External Interface Requirements
- 6. Data Requirements
- 7. System Constraints
- 8. Assumptions
- 9. Future Scope
- 10. Traceability Matrix

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) documents the functional and non-functional requirements of **Trace**, an AI-powered multi-camera person tracking and re-identification system. It is intended as a complete reference for the system's current implemented behaviour and serves as the basis for the SE Lab Assignment 2

submission.

1.2 Scope

Trace accepts CCTV video uploads from up to three active cameras (C01, C02, C03), processes them through a multi-stage AI pipeline (detection → tracking → crop extraction → embedding generation), and enables an operator to search for a specific person across all cameras using either a reference photograph or a previously registered person record. The system reconstructs the person's cross-camera route with timestamps, displays results on a real-time dashboard, and generates alerts when watchlisted persons are detected.

The system is a single-operator, single-server application with no authentication. It processes pre-recorded video files in batch mode — it does not perform live video stream processing.

1.3 Intended Audience

- Software engineering course evaluators
- Developers maintaining or extending the system
- Students studying the system architecture

1.4 Product Overview

Trace consists of three layers:

1. **Frontend** — A single-page vanilla JavaScript dashboard served by the backend, providing video upload, person registration, Re-ID search, route visualisation, and analytics views.
2. **Backend API** — A FastAPI application exposing REST endpoints and a WebSocket channel for real-time progress events.
3. **AI Pipeline** — A set of Python scripts and service wrappers performing person detection (YOLOv8n), multi-object tracking (BoT-SORT), quality-filtered crop extraction, and appearance embedding generation using OSNet x1_0 with optional SOLIDER, face (SCRFD + ArcFace), and KPR part-based models.

2. Overall Description

2.1 Product Perspective

Trace is a self-contained desktop/server application. It runs as a single Uvicorn process serving both the API and the static frontend. Data is persisted to a local SQLite database (`db/trace.db`) and JSON files on disk (`dataset/`). There is no dependency on external services at runtime beyond optional model weight downloads on first startup.

2.2 Product Functions (Summary)

Function	Status
Upload CCTV video per camera	Implemented
Validate camera ID and video format	Implemented
Person detection (YOLOv8n)	Implemented

Function	Status
Multi-object tracking (BoT-SORT)	Implemented
Quality-filtered crop extraction	Implemented
OSNet body embedding generation	Implemented
SOLIDER body embedding (optional)	Implemented (optional, requires weights)
Face embedding (SCRFD + ArcFace, optional)	Implemented (optional, requires weights)
KPR part-based embedding (optional)	Implemented (optional, requires weights)
Register person with watchlist status	Implemented
Enroll reference photo embedding	Implemented
Submit search query (photo or registered person)	Implemented
Query image auto-crop to largest person	Implemented
Cross-camera matching with triple fusion	Implemented
No-confident-match threshold	Implemented
Spatial-temporal route reconstruction	Implemented
Persist sightings and route steps	Implemented
Watchlist alert generation	Implemented
WebSocket real-time progress events	Implemented
Dashboard analytics	Implemented
Camera configuration (location, start time, transit)	Implemented
Authentication / multi-user	Not implemented
Live video stream processing	Not implemented
Cameras C04, C05	Not implemented (disabled in config)

2.3 User Classes

User Class	Description
Operator	The sole user. Uploads video, registers persons, submits search queries, views results and alerts. No authentication is required.

There is no separate administrator role. All operations are available to any user who can reach the server.

2.4 Operating Environment

Component	Requirement
OS	Windows 10/11, macOS, Linux
Python	3.9+ (3.11 recommended)
RAM	4 GB minimum, 8 GB recommended for KPR
GPU	Optional (NVIDIA CUDA); CPU-only is supported
Database	SQLite via aiosqlite
Server	Uvicorn (ASGI)
Browser	Any modern browser (Chrome, Firefox, Edge, Safari)

2.5 Design Constraints

- Single-process server (Uvicorn with reload in development).
- SQLite database — single-writer limitation.
- Upload job state is stored in-memory (lost on server restart).
- Only MVP cameras C01, C02, C03 are active. C04 and C05 are defined in configuration but disabled.
- KPR embedding requires a separate subprocess due to package name conflict with the standard torchreid library.
- Video formats limited to `.mp4`, `.avi`, `.mov`, `.mkv`.
- Image formats limited to `.jpg`, `.jpeg`, `.png`, `.webp`.

2.6 Assumptions and Dependencies

- Camera videos are pre-recorded files, not live streams.
- Each camera has a configured wall-clock start time so frame timestamps map to real-world time.
- The camera physical topology (adjacency graph and transit times) is known and configured in `camera_graph.json`.
- OSNet pretrained weights are downloaded automatically on first run (requires internet once).
- Optional models (face, KPR, SOLIDER) require manual weight downloads.

3. Functional Requirements

FR-01: Video Upload

The system shall allow the operator to upload a video file (MP4, AVI, MOV, or MKV) associated with a specific camera (C01, C02, or C03).

Implementation: `backend/app/routers/upload.py` — `POST /api/upload/video`

FR-02: Video/Camera Input Validation

The system shall reject uploads with an invalid camera ID (not in {C01, C02, C03}) or an unsupported video file extension, returning an appropriate error message.

Implementation: `backend/app/routers/upload.py` — validation checks before file save.

FR-03: Person Detection

The system shall detect all persons in each video frame using YOLOv8n with a configurable confidence threshold (default 0.6, COCO class 0 only).

Implementation: `ai_pipeline/detection/detect.py`, invoked via `backend/app/services/detection_service.py`.

FR-04: Multi-Object Tracking

The system shall assign persistent track IDs to detected persons within a camera video using the BoT-SORT tracker (via Ultralytics), producing per-frame track records with bounding boxes and timestamps.

Implementation: `ai_pipeline/tracking/track.py`, invoked via `backend/app/services/tracker_service.py`.

FR-05: Quality-Filtered Crop Extraction

The system shall extract person crop images from tracked bounding boxes, applying quality filters (minimum width 40 px, minimum height 100 px, severe edge truncation rejection), and produce a metadata JSON file listing all accepted crops with camera, track, frame, timestamp, and bounding box information.

Implementation: `ai_pipeline/reid/crop_extractor.py` with `quality_filter.py` and `sampling.py`, invoked via `backend/app/services/crop_service.py`.

FR-06: Body Embedding Generation (OSNet)

The system shall generate a 512-dimensional appearance embedding for each quality-filtered crop using OSNet x1_0 and store the gallery as a JSON file (`embeddings_<camera_id>.json`).

Implementation: `ai_pipeline/reid/embed.py`, invoked via `backend/app/services/embedding_service.py` (batch mode).

FR-07: Optional Face Embedding Generation

When SCRFD and ArcFace ONNX weights are configured, the system shall detect faces in crops and generate 512-dimensional face embeddings stored as `face_embeddings_<camera_id>.json`. If weights are absent, this step is skipped gracefully.

Implementation: `ai_pipeline/reid/face_embed.py`, invoked from `backend/app/routers/upload.py`.

FR-08: Optional KPR Part-Based Embedding Generation

When KPR weights and configuration are available, the system shall generate part-based embeddings (holistic + 8 body parts × 512-d) stored as `kpr_embeddings_<camera_id>.json`. Runs in a fresh subprocess due to package conflict. Skipped gracefully if unavailable.

Implementation: `ai_pipeline/reid/embed_kpr.py`, invoked from `backend/app/routers/upload.py` via subprocess.

FR-09: Person Registration

The system shall allow the operator to register a person with a name, optional alias, optional description, and a watchlist status (none, missing, suspect, or poi).

Implementation: `backend/app/routers/persons.py` — `POST /api/persons`.

FR-10: Reference Photo Enrollment

The system shall allow the operator to upload a reference photo for a registered person and generate an OSNet embedding stored with the person record. Re-enrollment replaces the previous embedding.

Implementation: `backend/app/routers/persons.py` — `POST /api/persons/{id}/enroll`.

FR-11: Search Query Submission

The system shall allow the operator to submit a person search query using either a registered person ID (uses stored embedding) or an uploaded reference image (embedded on the fly). Only one query may run at a time; concurrent submissions are rejected with HTTP 409.

Implementation: `backend/app/routers/queries.py` — `POST /api/queries`.

FR-12: Query Image Auto-Crop

When a multi-person image is submitted as a query, the system shall automatically detect persons using YOLOv8 and crop to the largest detected person before embedding. If no person is detected, the original image is used.

Implementation: `backend/app/services/pipeline_service.py` — `_autocrop_query_person()`.

FR-13: Cross-Camera Matching

The system shall compare the query embedding against all available camera galleries using cosine similarity, aggregate results per track (max similarity, mean top-3), and return up to 3 candidate tracks per camera that exceed the no-match threshold.

Implementation: `backend/app/services/matching_service.py` — `search_camera_top_k()`.

FR-14: Triple Fusion Matching

When face and/or KPR galleries are available, the system shall fuse body, face, and KPR similarity signals per track using configurable weights (default: body 0.40, face 0.30, KPR 0.60), re-normalised by active signal count.

Implementation: `backend/app/services/matching_service.py` — `_effective_score()`.

FR-15: No-Confident-Match Threshold

The system shall reject tracks whose effective (fused) score falls below the no-match threshold (default 0.74). If no track in any camera exceeds the threshold, the route is empty and the system reports no confident match.

Implementation: `backend/app/services/matching_service.py` — threshold check in `search_camera_top_k()`.

FR-16: Route Reconstruction

The system shall reconstruct an ordered cross-camera route from per-camera candidate matches using a greedy algorithm that fuses appearance score with spatial plausibility (camera adjacency graph) and temporal plausibility (Gaussian transit-time model). Fusion weights: appearance 0.60, spatial 0.25, temporal 0.15.

Implementation: `backend/app/services/route_service.py` — `reconstruct_route()`.

FR-17: Sighting and Route Persistence

The system shall persist one Sighting record per matched camera and ordered RouteStep records for the reconstructed route in the SQLite database.

Implementation: `backend/app/services/pipeline_service.py` — `_persist_sighting()`, RouteStep creation.

FR-18: Watchlist Alert Generation

When a query matches a person whose watchlist status is not "none", the system shall generate an Alert record with severity HIGH (for missing/suspect) or MEDIUM (for poi) and push it via WebSocket.

Implementation: `backend/app/services/pipeline_service.py` — `_fire_alert()`.

FR-19: Real-Time WebSocket Progress

The system shall push progress events (`query_progress`, `sighting_found`, `route_complete`, `alert`, `error`) to all connected WebSocket clients throughout query execution.

Implementation: `backend/app/core/websocket_manager.py`, `backend/app/services/pipeline_service.py`.

FR-20: Route Result Retrieval

The system shall provide an endpoint to retrieve the completed route for a query session, including ordered steps with camera location, timestamp, confidence, fusion score, and crop image path.

Implementation: `backend/app/routers/queries.py` — `GET /api/queries/{id}/route`.

FR-21: Dashboard Analytics

The system shall provide aggregated statistics: total persons, watchlist count, total/completed queries, total sightings, total/unacknowledged alerts, active cameras, recent alerts, and per-camera activity.

Implementation: `backend/app/routers/analytics.py` — `GET /api/analytics/dashboard`.

FR-22: Camera Configuration

The system shall allow the operator to update a camera's location name, recording start time, and transit times to neighbouring cameras. Changes are persisted to both the database and `camera_graph.json`.

Implementation: `backend/app/routers/cameras.py` — `PATCH /api/cameras/{id}`.

FR-23: Alert Acknowledgement

The system shall allow the operator to mark an alert as acknowledged.

Implementation: `backend/app/routers/analytics.py` — `POST /api/analytics/alerts/{id}/acknowledge`.

FR-24: Upload Job Status Polling

The system shall provide endpoints to poll the status of a video upload job and list all jobs.

Implementation: `backend/app/routers/upload.py` — `GET /api/upload/status/{job_id}`, `GET /api/upload/jobs`.

4. Non-Functional Requirements

NFR-01: Performance

Video processing time depends on hardware. On CPU, a 35-second video takes approximately 6–11 minutes (body only). GPU reduces this to 1–2 minutes. The system runs pipeline stages in background tasks or subprocesses to avoid blocking the API event loop.

NFR-02: Reliability

Optional pipeline stages (face, KPR, SOLIDER) catch exceptions and degrade gracefully — a failing optional signal never crashes the pipeline or blocks the upload job. The body (OSNet) gallery is always produced.

NFR-03: Usability

The frontend is a single-page dashboard with sidebar navigation covering: Dashboard, Re-ID Search Query, Route Map, Watchlist & Persons, Pipeline Video Studio, and Camera Configuration. Real-time WebSocket updates eliminate the need for manual polling.

NFR-04: Maintainability

The backend follows a layered architecture: routers (HTTP handlers) → services (business logic) → ai_pipeline scripts (ML inference). Each service wraps a single pipeline script via subprocess, keeping ML code isolated from web framework code.

NFR-05: Extensibility

The matching service is dimension-agnostic (works with any embedding dimension). New Re-ID backbones can be added by producing a gallery JSON file in the expected format. The camera graph is data-driven — adding cameras requires only configuration changes.

NFR-06: Security

The system currently implements no authentication or authorization. CORS is configured to allow all origins (development mode). This is documented as a constraint, not a feature.

NFR-07: Availability

The system is designed for single-operator use. One query runs at a time by design (enforced with HTTP 409). Upload jobs run sequentially in background tasks.

5. External Interface Requirements

5.1 User Interface

A single-page web application served at `http://localhost:8000`. Views include:

- **Dashboard** — Analytics stats, recent alerts, camera activity
- **Re-ID Search Query** — Submit search by photo or registered person
- **Route Map** — SVG floor-plan with animated route visualisation
- **Watchlist & Persons** — Register/manage persons, enroll photos
- **Pipeline Video Studio** — Upload video per camera, view job progress
- **Camera Configuration** — Edit camera location, start time, transit times

5.2 REST API

Base URL: `/api`. Full OpenAPI documentation at `/api/docs`.

Method	Endpoint	Description
POST	<code>/api/upload/video</code>	Upload video, trigger pipeline
GET	<code>/api/upload/jobs</code>	List all upload jobs
GET	<code>/api/upload/status/{job_id}</code>	Poll single job status
POST	<code>/api/queries</code>	Submit a Re-ID search query
GET	<code>/api/queries/{id}</code>	Poll query session status
GET	<code>/api/queries/{id}/route</code>	Get completed route result
GET	<code>/api/queries</code>	List recent query sessions
GET	<code>/api/cameras</code>	List all cameras with graph
GET	<code>/api/cameras/{id}</code>	Get single camera detail
PATCH	<code>/api/cameras/{id}</code>	Update camera config
GET	<code>/api/cameras/{id}/sightings</code>	Recent sightings for camera
GET	<code>/api/persons</code>	List registered persons
GET	<code>/api/persons/{id}</code>	Get single person
POST	<code>/api/persons</code>	Register a new person
PUT	<code>/api/persons/{id}</code>	Update person fields
DELETE	<code>/api/persons/{id}</code>	Soft-delete person
POST	<code>/api/persons/{id}/enroll</code>	Enroll reference photo
POST	<code>/api/persons/search</code>	Image-based person lookup

Method	Endpoint	Description
GET	/api/analytics/dashboard	Dashboard stats
GET	/api/analytics/alerts	List alerts
POST	/api/analytics/alerts/{id}/acknowledge	Acknowledge alert
GET	/api/analytics/sightings/recent	Recent sightings timeline
GET	/api/health	Health check

5.3 WebSocket Interface

Endpoint: `ws://localhost:8000/ws`

Event Type	Direction	Payload
query_progress	Server → Client	session_id, progress_pct, message
sighting_found	Server → Client	session_id, sighting object
route_complete	Server → Client	session_id, full route object
alert	Server → Client	alert object
error	Server → Client	session_id, error message

5.4 File/Video Interface

- Video uploads: multipart/form-data, saved to `dataset/raw_videos/`
- Reference photos: saved to `dataset/uploads/reference_photos/`
- Query images: saved to `dataset/uploads/query_images/`
- Crop images: stored in `dataset/crops/<camera_id>/`

5.5 Database Interface

SQLite via SQLAlchemy 2.0 async (aiosqlite driver). Database file: `db/trace.db`. Schema auto-created on startup.

5.6 AI Model Interfaces

Model	Framework	Input	Output
YOLOv8n	Ultralytics/PyTorch	Video frames	Person bounding boxes
BoT-SORT	Ultralytics	Detections per frame	Track IDs
OSNet x1_0	torchreid/PyTorch	256×128 crop image	512-d embedding
SOLIDER Swin-Small	PyTorch/timm	256×128 crop image	768-d embedding
SCRFD 10G	ONNX Runtime	Crop image	Face bounding boxes

Model	Framework	Input	Output
ArcFace ResNet-50	ONNX Runtime	Aligned face	512-d embedding
KPR Swin-Small	PyTorch	Crop image + keypoints	Holistic + 8×512-d part embeddings

6. Data Requirements

6.1 Database Entities

Entity	Table	Key Fields
Person	persons	id, name, alias, description, watchlist_status, reference_embedding (JSON), reference_image_path, is_active, created_at
Camera	cameras	id (PK, text), location, description, start_time, is_active
QuerySession	query_sessions	id, person_id (FK), query_image_path, status, progress_pct, submitted_at, completed_at, error_message
Sighting	sightings	id, session_id (FK), camera_id (FK), track_id, first_seen, last_seen, best_confidence, mean_confidence, crop_path, appearance_score, spatial_score, temporal_score, fusion_score
RouteStep	routes	id, session_id (FK), step_order, sighting_id (FK)
Alert	alerts	id, session_id (FK), person_id (FK), sighting_id (FK), severity, title, message, acknowledged, created_at

6.2 File-Based Data Stores

Store	Format	Content
dataset/embeddings_<cam>.json	JSON array	Body embedding records per crop
dataset/face_embeddings_<cam>.json	JSON array	Face embedding records per crop
dataset/kpr_embeddings_<cam>.json	JSON array	KPR part embedding records per crop
dataset/crops_metadata_<cam>.json	JSON array	Crop metadata (path, camera, track, frame, timestamp, bbox)
dataset/camera_graph.json	JSON object	Camera topology, adjacency, transit times

Store	Format	Content
dataset/crops/<cam>/	JPEG files	Extracted person crop images
dataset/raw_videos/	Video files	Uploaded camera videos

6.3 Configuration Data

File	Content
.env	Runtime settings (model paths, thresholds, fusion weights)
ai_pipeline/config.yaml	Detection params, camera start times, ReID config, quality thresholds

7. System Constraints

1. **No authentication** — Any network-reachable user has full access.
2. **Single query at a time** — Concurrent queries are rejected (HTTP 409).
3. **In-memory upload job store** — Job status is lost on server restart.
4. **SQLite single-writer** — Limits concurrent write throughput.
5. **Three active cameras only** — C04 and C05 are defined but disabled.
6. **Batch processing only** — No live/streaming video support.
7. **KPR subprocess isolation** — KPR cannot run in-process with OSNet due to package name conflict.
8. **CPU fallback** — All models run on CPU when no GPU is available, with significant performance degradation for KPR.

8. Assumptions

1. The physical camera layout and approximate walking transit times between cameras are known in advance and encoded in `camera_graph.json`.
2. Each camera video's wall-clock start time is known and configured.
3. Persons in the CCTV footage are visible enough for detection (minimum crop size thresholds apply).
4. The system operates in a controlled environment (e.g., a building with fixed cameras), not arbitrary outdoor scenes.
5. The no-match threshold (0.74) was determined empirically on a small validation set and is not a universally calibrated value.

9. Future Scope

The following are identified as potential future enhancements. They are **not implemented** in the current system:

Feature	Status
User authentication and role-based access	Planned
Live video stream processing	Planned
Activation of cameras C04 and C05	Planned (configuration exists)
Persistent upload job queue (survives restart)	Planned
Multi-query concurrency	Planned
Database migration to PostgreSQL	Planned
Mobile-responsive UI improvements	Planned
Automated threshold calibration with larger datasets	Planned

10. Traceability Matrix

Req ID	System Component	Implementation File(s)	Diagram Ref
FR-01	Upload Router	backend/app/routers/upload.py	DFD L1 §1.0, Seq
FR-02	Upload Router	backend/app/routers/upload.py	DFD L1 §1.0
FR-03	Detection Service	ai_pipeline/detection/detect.py	DFD L2 §1.1, backend/app/services/detect
FR-04	Tracker Service	ai_pipeline/tracking/tracker.py	DFD L2 §1.2, s/tracker_
FR-05	Crop Service	ai_pipeline/reid/crop_extractor.py	DFD L2 §1.3, backend/app/services/cro
FR-06	Embedding Service	ai_pipeline/reid/embed.py	DFD L2 §1.4, bedding_se
FR-07	Upload Router	ai_pipeline/reid/face_embed.py	DFD L2 §1.5, backend/app/routers/upload.p
FR-08	Upload Router	ai_pipeline/reid/embed_ko	DFD L2 §1.6, /upload.py
FR-09	Persons Router	backend/app/routers/persons.py	Use Case, DFD L1
FR-10	Persons Router	backend/app/routers/person	Use Case
FR-11	Queries Router	backend/app/routers/queries.py	DFD L1 §4.0
FR-12	Pipeline Service	backend/app/services/pipeline.py	DFD L2 §4.1, Activity
FR-13	Matching Service	backend/app/services/matching_service.py	DFD L2 §4.2
FR-14	Matching Service	backend/app/services/match	DFD L2 §4.3
FR-15	Matching Service	backend/app/services/matching_service.py	DFD L2 §4.3
FR-16	Route Service	backend/app/services/route	DFD L2 §5.1, Activity
FR-17	Pipeline Service	backend/app/services/pipeline_service.py	Seq

Req ID	System Component	Implementation File(s)	Diagram Ref
FR-18	Pipeline Service	backend/app/services/pipe	Seq, Use Case
FR-19	WebSocket Manager	backend/app/core/websocket	Sequence Diagram
FR-20	Queries Router	backend/app/routers/queri	Seq
FR-21	Analytics Router	backend/app/routers/analyt	Use Case
FR-22	Cameras Router	backend/app/routers/camer	Use Case
FR-23	Analytics Router	backend/app/routers/analyt	Use Case
FR-24	Upload Router	backend/app/routers/uploa	DFD L1

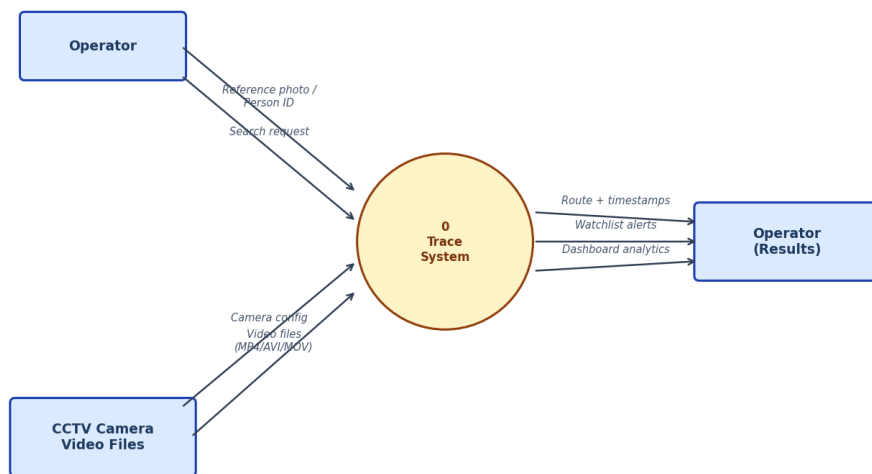
2. Data Flow Diagrams (DFD)

These DFD diagrams model data flow through the Trace system using standard notation: rectangles for external entities, circles for processes, open-ended rectangles for data stores, and arrows for data flows. All diagrams are derived from the actual implemented architecture.

2.1 DFD Level 0 - Context Diagram

Figure 1: DFD Level 0 - Context Diagram

DFD Level 0 — Context Diagram: Trace System

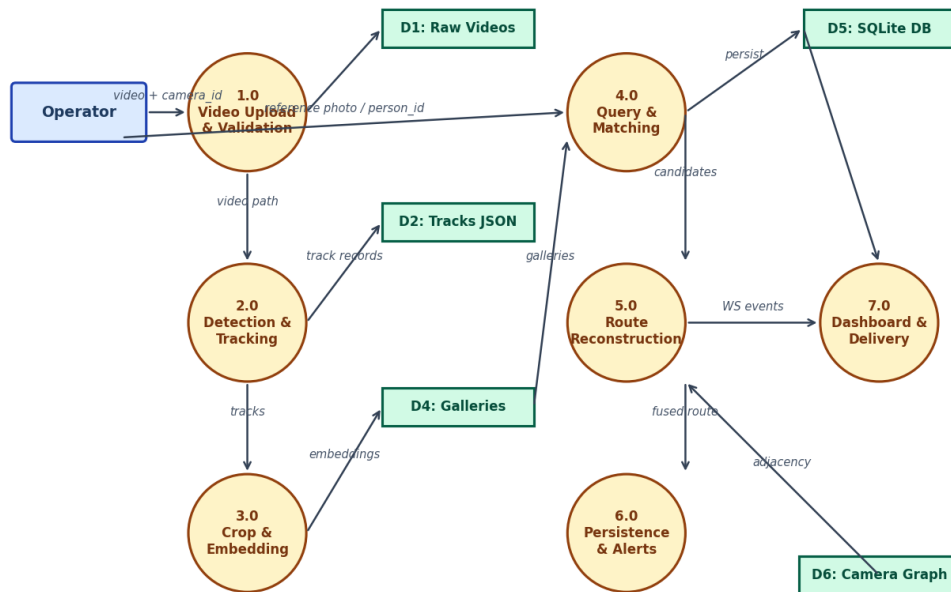


The context diagram shows Trace as a single process (Process 0) interacting with two external entities: the Operator (who submits search queries, reference photos, and camera configuration) and CCTV Camera Video Files (uploaded for processing). Outputs include reconstructed routes with timestamps, watchlist alerts, and dashboard analytics.

2.2 DFD Level 1 - System Decomposition

Figure 2: DFD Level 1 - Major Process Decomposition

DFD Level 1 — Trace System Decomposition

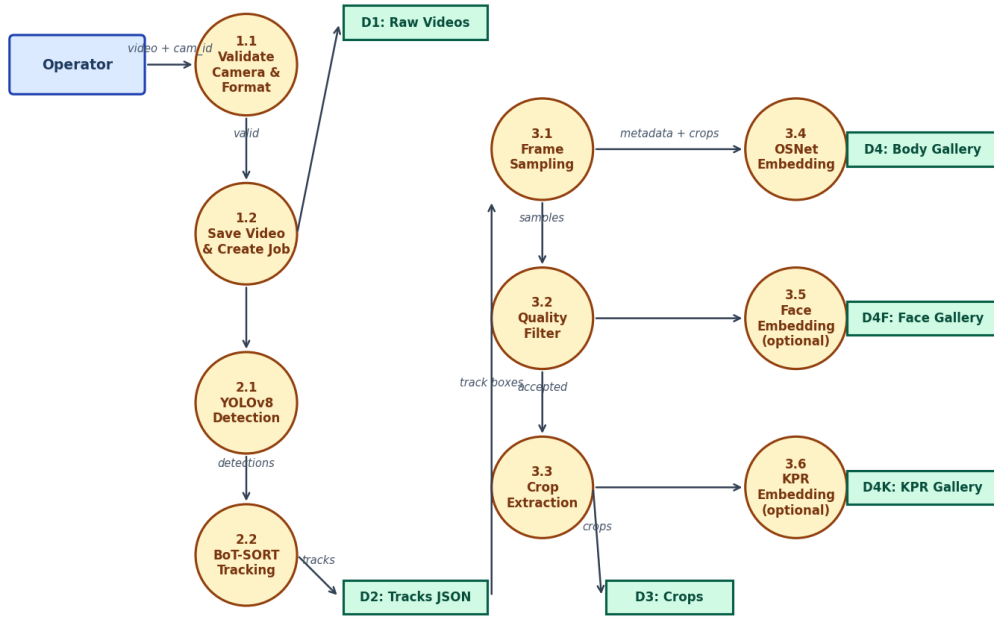


Level 1 decomposes Trace into seven processes: (1) Video Upload and Validation, (2) Detection and Tracking, (3) Crop Extraction and Embedding, (4) Query Submission and Matching, (5) Route Reconstruction, (6) Persistence and Alerts, and (7) Dashboard and Result Delivery. Data stores: D1 (raw videos), D2 (tracks JSON), D4 (embedding galleries), D5 (SQLite DB), D6 (camera graph).

2.3 DFD Level 2 - Video Processing Pipeline

Figure 3: DFD Level 2 - Video Processing (Processes 1.0-3.0)

DFD Level 2 — Video Processing Pipeline (Processes 1.0-3.0)

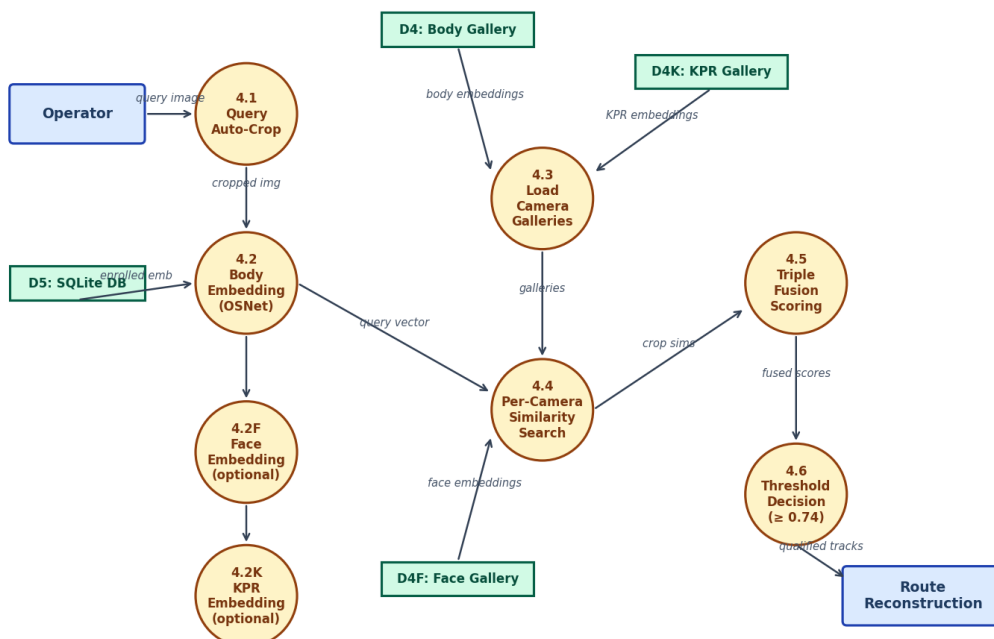


Expands the video processing pipeline. Process 1.1 validates camera_id (C01/C02/C03) and format (MP4/AVI/MOV/MKV). Process 2.1 runs YOLOv8 detection; 2.2 runs BoT-SORT tracking. Process 3.1 samples frames per track; 3.2 applies quality filtering (min 40x100px, edge truncation); 3.3 extracts crop JPEGs; 3.4 generates OSNet embeddings (always); 3.5/3.6 generate face/KPR embeddings (optional, weight-dependent).

2.4 DFD Level 2 - Person Re-ID / Matching

Figure 4: DFD Level 2 - Query and Matching (Process 4.0)

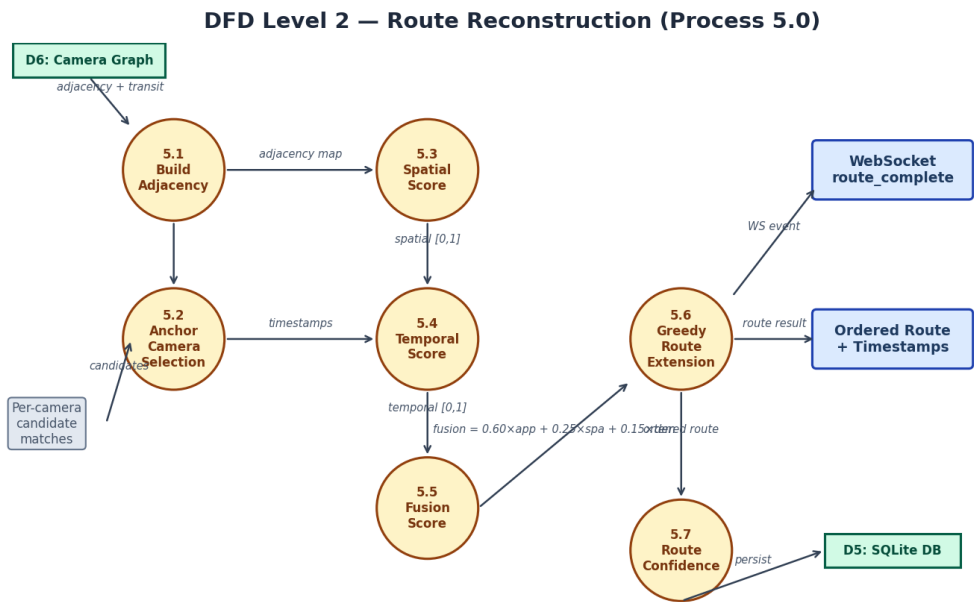
DFD Level 2 — Person Re-ID / Matching (Process 4.0)



Expands the matching process. The query image is auto-cropped to the largest person (4.1), body embedding via OSNet (4.2), optional face (4.2F) and KPR (4.2K) embeddings. Galleries are loaded per camera (4.3), cosine similarity computed per crop (4.4), triple fusion scoring applied (4.5), and threshold decision at 0.74 filters tracks (4.6). Tracks below threshold produce "no confident match."

2.5 DFD Level 2 - Route Reconstruction

Figure 5: DFD Level 2 - Route Reconstruction (Process 5.0)

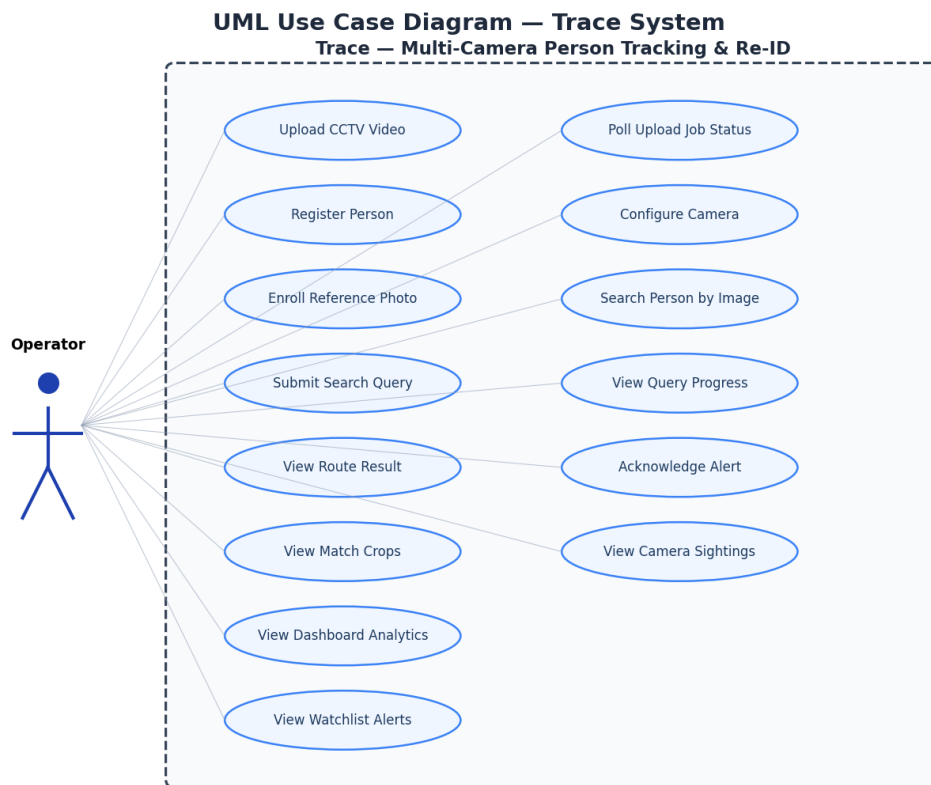


Route reconstruction builds camera adjacency from camera_graph.json (5.1), selects anchor camera (5.2), computes spatial score (1-hop=1.0, 2-hop=0.4, else 0.0) (5.3), temporal score via Gaussian centred on expected transit time (5.4), and fuses: 0.60 x appearance + 0.25 x spatial + 0.15 x temporal (5.5). Greedy extension iteratively picks the best camera-track pair (5.6). Route confidence is the mean fusion score (5.7).

3. UML Diagrams

3.1 Use Case Diagram

Figure 6: UML Use Case Diagram

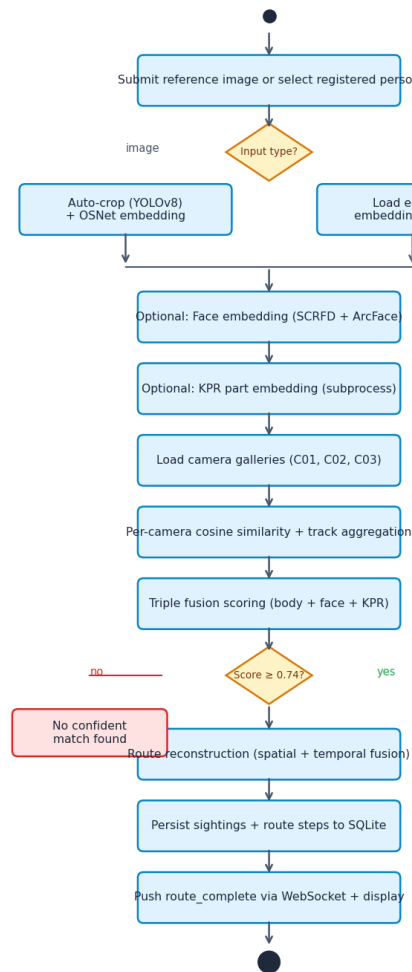


One primary actor: the Operator. Use cases span five areas: Video Processing (upload, validate, poll), Person Management (register, enroll, update, delete, search by image), Re-ID Query (submit, progress, route, crops), Monitoring (dashboard, alerts, acknowledge, sightings), and Configuration (camera settings). No admin role exists in the implementation - all operations are available to the single Operator actor.

3.2 Activity Diagram

Figure 7: UML Activity Diagram - Person Search and Route Reconstruction

UML Activity Diagram — Person Search & Route Reconstruction

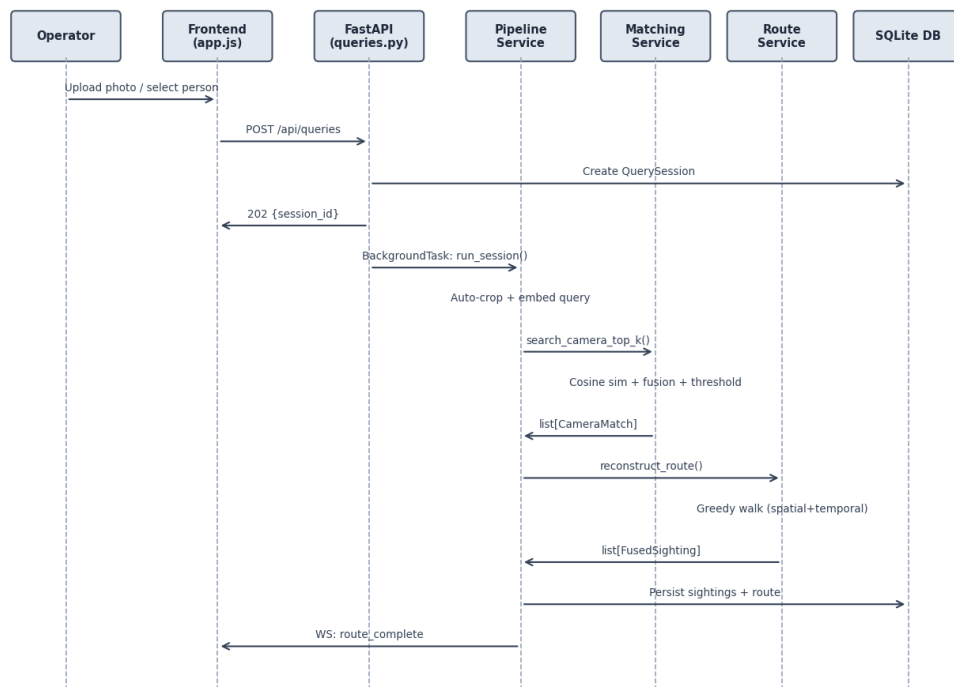


Shows the complete search workflow. Decision points: input type (image vs. registered person), optional signal availability (face, KPR), the 0.74 threshold check, and whether any camera produces candidates. The no-confident-match branch is explicitly modelled. Route reconstruction uses greedy extension with spatial-temporal fusion. Watchlist alert generation is conditional on the person's watchlist_status.

3.3 Sequence Diagram

Figure 8: UML Sequence Diagram - Search Person Across Cameras

UML Sequence Diagram — Search Person Across Cameras

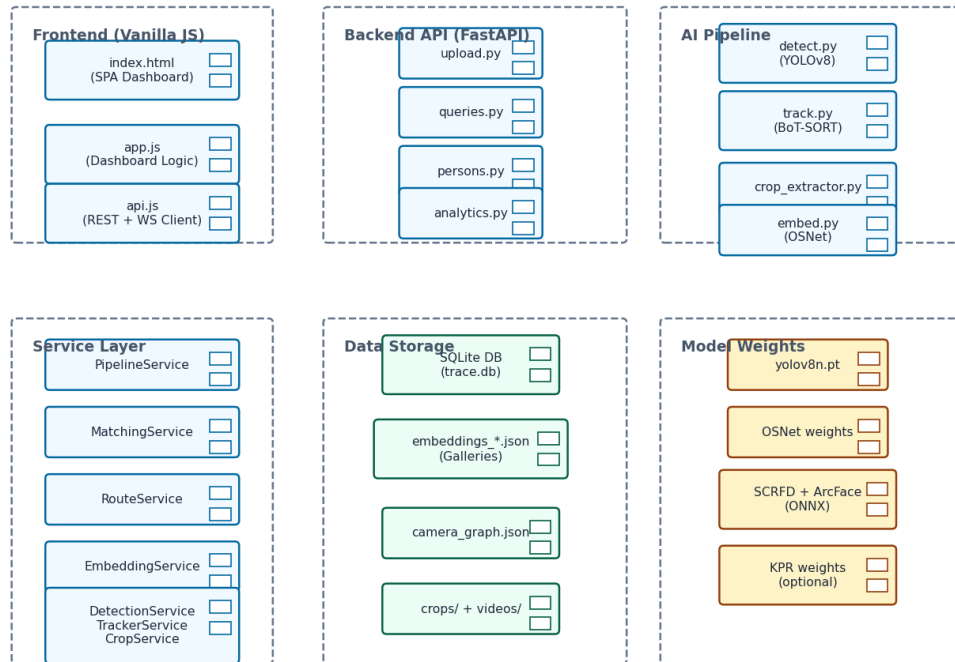


Traces the search workflow through actual components: Operator, Frontend (app.js), FastAPI (queries.py), PipelineService, MatchingService, RouteService, SQLite DB, and WebSocketManager. Key interactions: 202 async response, background task execution, per-camera matching loop with triple fusion, greedy route reconstruction, DB persistence of sightings and route steps, and WebSocket broadcast of route_complete event.

3.4 Component Diagram

Figure 9: UML Component Diagram - System Architecture

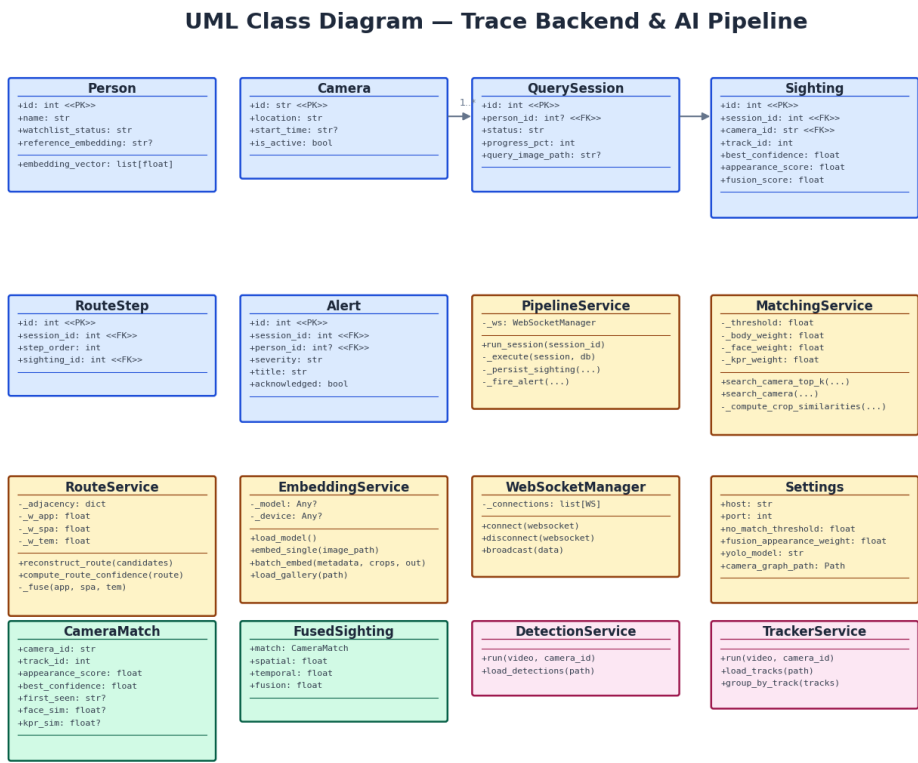
UML Component Diagram — Trace System Architecture



Shows physical architecture: Frontend (vanilla JS SPA), Backend API (FastAPI with 5 routers + WebSocket), Service Layer (Pipeline, Matching, Route, Embedding, Detection, Tracker, Crop), AI Pipeline scripts, Data Storage (SQLite, JSON galleries, camera graph, crops/videos), and Model Weights (YOLOv8, OSNet, SCRFD/ArcFace ONNX, KPR optional).

4. Class Diagram

Figure 10: UML Class Diagram - Backend and AI Pipeline



Database entities (Person, Camera, QuerySession, Sighting, RouteStep, Alert) are SQLAlchemy ORM models with PK/FK relationships. Service classes implement business logic: PipelineService orchestrates the query session; MatchingService performs triple-fusion similarity search; RouteService does spatial-temporal route reconstruction; EmbeddingService wraps OSNet inference; WebSocketManager broadcasts events. Data classes CameraMatch and FusedSighting carry intermediate results. Settings centralises configuration. Key relationships: QuerySession composes Sightings and RouteSteps (cascade delete); Sighting references Camera; Person optionally links to QuerySession and Alert.

5. Traceability and Consistency Table

Traceability Matrix

Requirement → Component → Implementation → Diagram

Req ID	Requirement Summary	System Component	Implementation File(s)	DFD Ref	UML Ref
FR-01	Video upload	Upload Router	backend/app/router/upload.py	DFD L1 \$1.0	Use Case UC1, Seq
FR-02	Input validation	Upload Router	backend/app/router/upload.py	DFD L1 \$1.0, DFD L2 \$1.1	Use Case UC2
FR-03	Person detection	Detection Service + AI Script	backend/app/service/detection_service.py	DFD L2 \$2.1	Component pipeline/detection
FR-04	Multi-object tracking	Tracker Service + AI Script	backend/app/service/tracker_service.py	DFD L2 \$2.2	Component pipeline/tracking
FR-05	Quality-filtered crop extraction	Crop Service + AI Script	backend/app/service/crop_service.py	DFD L2 \$2.3	Component pipeline/reid/crop
FR-06	OSNet body embedding	Embedding Service + AI Script	backend/app/service/embedding_service.py	DFD L2 \$3.4	Component pipeline/reid/body
FR-07	Face embedding (optional)	Upload Router + AI Script	backend/app/router/upload.py, ai_pipeline/reid/face_embed.py	DFD L2 \$3.5	Component pipeline/reid/face_embed
FR-08	KPR embedding (optional)	Upload Router + AI Script	backend/app/router/upload.py, ai_pipeline/reid/kpr_embed.py	DFD L2 \$3.6	Component pipeline/reid/kpr_embed
FR-09	Person registration	Persons Router	backend/app/router/persons.py	DFD L1	Use Case UC4
FR-10	Reference photo enrollment	Persons Router + EmbeddingService	backend/app/router/persons.py	DFD L1	Use Case UC5, es/embedding
FR-11	Search query submission	Queries Router	backend/app/router/queries.py	DFD L1	Use Case UC9, Seq
FR-12	Query image auto-crop	PipelineService	backend/app/service/pipeline_service.py	DFD L2 \$4.1	rv Activity, Seq
FR-13	Cross-camera matching	MatchingService	backend/app/service/matching_service.py	DFD L2 \$4.2	rv Activity, Seq
FR-14	Triple fusion matching	MatchingService	backend/app/service/matching_service.py	DFD L2 \$4.5	rv Activity
FR-15	No-confident-match threshold	MatchingService	backend/app/service/matching_service.py	DFD L2 \$4.6	rv Activity

Req ID	Requirement Summary	System Component	Implementation File(s)	DFD Ref	UML Ref
FR-16	Route reconstruction	RouteService	backend/app/service/route_service.py	DFD L2 §5.1–5.7	Activity, Seq
FR-17	Sighting/route persistence	PipelineService	backend/app/service/pipeline_service.py	DFD L1 §6.0	Seq
FR-18	Watchlist alert generation	PipelineService	backend/app/service/pipeline_service.py	DFD L1 §6.0	Seq, Use Case UC14
FR-19	WebSocket real-time progress	WebSocketManager	backend/app/core/websocket_manager.py	DFD L1 §6.0	Seq, Component
FR-20	Route result retrieval	Queries Router	backend/app/router/queries.py	DFD L1 §7.0	Use Case UC11
FR-21	Dashboard analytics	Analytics Router	backend/app/router/analytics.py	DFD L1 §7.0	Use Case UC13
FR-22	Camera configuration	Cameras Router	backend/app/router/cameras.py	DFD L1	Use Case UC17
FR-23	Alert acknowledgement	Analytics Router	backend/app/router/analytics.py	DFD L1 §7.0	Use Case UC15
FR-24	Upload job status polling	Upload Router	backend/app/router/upload.py	DFD L1 §1.0	Use Case UC3

Diagram Cross-Reference

Diagram	File	Requirements Covered
DFD Level 0 (Context)	DFD_Level_0.mmd	All (system boundary)
DFD Level 1	DFD_Level_1.mmd	FR-01–FR-24 (process decomposition)
DFD Level 2 — Video Processing	DFD_Level_2_Video_Processing.mmd	FR-01–FR-08
DFD Level 2 — Matching	DFD_Level_2_Matching.mmd	FR-11–FR-15
DFD Level 2 — Route	DFD_Level_2_Route.mmd	FR-16–FR-19
UML Use Case	UML_Use_Case.puml	FR-01, FR-09–FR-11, FR-18, FR-20–FR-24
UML Activity	UML_Activity.puml	FR-11–FR-18
UML Sequence	UML_Sequence.puml	FR-11–FR-20
UML Component	UML_Component.puml	All (architectural view)
UML Class	UML_Class.puml	All (structural view)

Verification Notes

- All requirements FR-01 through FR-24 are verified as **IMPLEMENTED** from source code inspection.
- No requirement references a feature that is absent from the codebase.
- Optional features (FR-07, FR-08) are explicitly marked as dependent on external model weights.
- The no-match threshold value (0.74) is sourced from `ai_pipeline/config.yaml` and `backend/app/config.py`.
- Fusion weights are sourced from `backend/app/config.py` (Settings class).

6. References

1. Trace Repository Source Code: backend/app/, ai_pipeline/, frontend/
2. README.md - Project documentation and tech stack
3. ai_pipeline/config.yaml - AI pipeline configuration
4. dataset/camera_graph.json - Camera topology and transit times
5. backend/app/config.py - Application settings (Settings class)
6. backend/app/models/orm.py - SQLAlchemy ORM definitions
7. backend/app/models/schemas.py - Pydantic request/response schemas
8. YOLOv8 (Ultralytics) - Person detection
9. BoT-SORT - Multi-object tracking (via Ultralytics)
10. OSNet x1_0 (torchreid) - Body Re-ID embedding (512-dim)
11. SCRFD 10G + ArcFace ResNet-50 - Face detection/recognition (ONNX)
12. KPR Swin-Small (ECCV 2024) - Part-based Re-ID
13. SOLIDER Swin-Small - Cloth-change robust embedding (768-dim)